

EPQ ARTEFACT REPORT

Creating a Web-based Origami Crease Pattern Generator

King's College London Mathematics School

Author

Shenglun Hutton

shenglun.hutton@kcl.ac.uk

Supervisor

Ms Esslemont

jenny.esslemont@kcl.ac.uk

Abstract

This artefact report details the development process and evaluation of my EPQ:
Creating a web-based origami crease pattern generator. The artefact is a computational tool that simplifies the design process of origami crease patterns based on user-defined specifications. The generator employs algorithms based on computational geometry and graph theory in order to find a suitable crease pattern for a tree input designed by the user.

Contents

1	Introduction	3
2	Literature Review	4
2.1	Defining Origami	4
2.2	Origami Design	5
2.3	Computational Origami	5
2.3.1	TreeMaker	5
2.3.2	JavaScript Rabbit Ear Library	6
3	Methodology	8
3.1	Timeline	8
3.2	Tools used	9
4	Development	10
4.1	Interactive Planar Graph	10
4.1.1	Setting up a canvas	10
4.1.2	Event handling	10
4.1.3	Node manipulation	10
4.1.4	Connections	11
4.1.5	User Menu	11
4.2	Convex Hull	12
4.2.1	Quickhull method	12
4.3	Circle Packing: Compacting circles	14
4.4	Finding major creases	17
4.4.1	Finding polygons	17
4.4.2	Active Paths	18
4.4.3	Other lines of interest	18
4.5	Finding sub-bases	19
4.5.1	Triangles	19
4.5.2	Quadrilaterals	21
4.6	Significant helper functions	23
4.6.1	Getting the shortest path	23
4.6.2	Removing duplicates in an array	24
4.6.3	Linking two canvas elements	25
5	Final Results	26

<i>EPQ Artefact Report</i>	2
5.1 Crease pattern generator: Walkthrough	26
5.2 Comparison with original goals	28
5.3 Comparison with TreeMaker	31
5.4 Final Tests	33
A Definitions	35

1 Introduction

Paper folding is an ancient art that has undergone countless transformations, the most prominent of which occurred in the last century. The term "origami" originally described Japanese paper folding only, however more modern usage has adapted it to encompass the intentional folding of any medium, from cell membranes to giant space mirrors. These technological achievements are the hard-earned product of the brief but dense history of modern origami.

Traditional origami methods relied heavily on trial and error: the artist would have chosen a starting point from a selection of existing bases then utilised their individual experience to fold, unfold and refold until the model was satisfactory. Similar to other forms of traditional art, this was a somewhat inefficient process, although it allowed for a creative freedom which is difficult to model with a computer. However, without a formal language with which to communicate plans and ideas, the art's development was greatly hindered.

The first origami manual Senbazuru Orikata to ever be published introduced the concept of diagramming, however some of the steps were unclear and the notation irregular and hard to follow [1]. The origami artist Akira Yoshizawa played an important role in resolving this issue. His published collections of origami designs had both pictorial and text-based notations, and proposed a formal system that could be applied to any origami design process. Soon after, it was extended by Samuel Randlett to create the now universal diagramming system: the Yoshizawa-Randlett notation system [2].

With the creation of this concise and descriptive language, origami became an elegant candidate to be modelled computationally. In fairly recent history, mathematicians were concerned with paper folding from a geometric perspective, and as technology advanced and origami became a formalised process, this perspective broadened into a new branch of computer science: computational origami. The origami design process is constantly being researched and revisioned, with countless new algorithms created to accommodate for increasingly complex models. In this project, I will be exploring one such algorithm, which is widely used to generate crease patterns: the tree method [3].

2 Literature Review

Much of the origami design process can be aided by mathematical ideas. Methods like polygon packing have been used extensively in this area of research, which I explored prior to writing any code in order to glean a better understanding of what the scope of my project should look like [4].

2.1 Defining Origami

Diagramming origami has become an art within of itself, and it has evolved alongside origami to extend its features to allow concise visual descriptions of many different folds, however the basic convention followed is that of the Yoshizawa-Randlett system [5].

This project doesn't account for the creation of a step-by-step folding process in which diagramming skill would be essential, however knowing the topic terminology was essential to understanding the research material I used later on. The article Origami Mathematics by Adrian Martin [6] provided a further overview on the conventions used in modern origami, as well as methods for checking whether a crease pattern is foldable or not.

Upon further reading, there exist several sets of axioms that a flat-foldable origami model follows, one of which is Huzita's Axioms:

1. Given two distinct points A and B, there is a unique fold that passes through both of them
2. Given two distinct points A and B, there is a unique fold that places A onto B
3. Given two lines M and N there is a fold that places M onto N
4. Given a point A and a line M, there is a unique fold perpendicular to M that passes through point A
5. Given two points A and B and a line M there is a fold that places A onto M and passes through B
6. Given two points A and B and two lines M and N, there is a fold that places A onto M and B onto N
7. Given one point A and two lines M and N, there is a fold that places

A onto M which is perpendicular to N These were useful as laws with which to hold my project against for error-checking, when debugging and for potential use in the user interface.

2.2 Origami Design

When considering origami design, Robert Lang's tree method is arguably the most widely used. It is described extensively in his book *Origami Design Secrets* [7] along with several articles and the *TreeMaker* executable written in C [3]. Throughout the project, this was the resource I turned to the most, and the underlying theory for much of the programming is all based on the methods described in this book. It provided the best insight into how I was going to structure the timeline of my project.

2.3 Computational Origami

Computational origami simplifies the geometric aspect of the design process. For example, circle packing - an important method used in crease generation - by hand is tedious work that requires trial and error, while a laptop is capable of computing the circle packing of thirty nodes in under a second.

2.3.1 TreeMaker

Robert Lang's *TreeMaker* [8] is a famous example of transferring a geometric method onto a digital platform. It implements the circle packing method on a square piece of origami paper which is represented as a planar graph of height and width 1.0. The user inputs are manual with regards to placing the nodes and adjusting the weight - the relative size - of the flaps and rivers, however calculating how to maximise the paper used and where the creases should lie is done automatically: circle packing, scale readjustment and triangulation to obtain the final crease pattern.

The layout of this program is transparent with the user, and was incredibly useful for understanding what information I would need from the user and how I needed to use it. I used this tool as a comparison against my work.

2.3.2 JavaScript Rabbit Ear Library

The Rabbit Ear library [9] is one of the most complete collections of origami methods in a digitally interactive form. In deciding the language to write the program with, I took a lot of inspiration from Robby Kraft's Learn pages for his various projects, as the fluid and dynamic user interface stood out. Similar to TreeMaker, the paper is represented by a planar graph, and the class `graph` is made of a finite number of vertices, edges and faces. I didn't end up using this library however, as I wanted to build my project manually in order to gain a better understanding of the maths behind origami.

3 Methodology

3.1 Timeline



06/09/2023	●	Research: Origami Design Secrets by Robert Lang & Rabbit Ear module by Robby Kraft
20/09/2023	●	Writeup: Initial Overleaf setup and Introduction & Literature Review first draft
22/09/2023	●	Proposal form and project title set
01/10/2023	●	Writeup: Methodology rough project timeline
15/10/2023	●	Code: Interactive planar graph & structure
20/10/2023	●	Code: Maths functions implemented
31/10/2023	●	Writeup: Methodology tools chosen
20/11/2023	●	Code: Getting a convex hull of terminal vertices
30/11/2023	●	Writeup: Development so far
25/12/2023	●	Code: Circle packing method
10/01/2024	●	Code: Major creases and polygon identification
15/01/2024	●	Writeup: Development so far
05/02/2024	●	Code: Sub-base creases for triangles and quadrilaterals
15/02/2024	●	Presentation: Make PowerPoint and script
25/02/2024	●	Code: Fix any errors and clean up code
10/03/2024	●	Writeup: Conclusions

3.2 Tools used

There were three programming languages that I considered using for this project, and although I had considered using a mixture I concluded that this wouldn't be necessary. Below is a brief summary of the points I considered.

	Pros	Cons
Python	Most experience with	Graphing capabilities are not suited for the purpose of this project
JavaScript	Designed for user interaction Rabbit Ear is written entirely in JavaScript Visuals are relatively easy to implement	No experience using
C++	TreeMaker is written in C++ Useful for complex applications as it has more functionalities	Limited experience using Complex language to learn

Table 1: Pros and Cons of programming languages

I decided to use JavaScript following this, as I was convinced by Robby Kraft's work that JavaScript could be used for origami crease generation.

Programming and writeup tools:

- Webstorm IDE: This is familiar as I have used JetBrains' PyCharm before, and furthermore can connect to GitHub, as well as being free to use for a student license
- GitHub: Useful for backing up code and keeping a log of all the changes
- Overleaf: For typesetting the artefact report
- OneNote: For organising all the research, planning and writeup sections as well as keeping track of the drafts
- PowerPoint: I used this program for the presentation

4 Development

4.1 Interactive Planar Graph

4.1.1 Setting up a canvas

The canvas and webpage layout is all written in HTML, and the JS script linked under the script tag. Initially I was planning on using the paper-script.js library in conjunction with canvas however it hadn't been updated to work with the latest version of JavaScript - at that point.

4.1.2 Event handling

There are three ways in which the user can interact with the canvas, where the computer recognises it as an input:

- Mouse down: initialises the canvas and checks for the region in the graph local to where the mouse is, determining which action should be taken depending on: which mouse button was clicked; whether or not a node is within the search radius of the click; and the state of that node. The graph is updated after every mouse down event.
- Mouse up: "cleans the slate" of whatever state the mouse is in and handles some checks that aren't in the mouse down event to prevent sequencing errors. The graph is also updated after every mouse up event.
- Mouse movement: mainly handles node movement, but also updates the convex hull.

4.1.3 Node manipulation

Adding

A new node is added upon a mouse down event where no node is found in the local vicinity of the mouse position, which updates the nodes array and the connections array.

I chose not to create a node class at the start of my project due to my lacking experience in JavaScript. However, as I progressed further, I realized that this decision did not affect the functionality of the initial functions, so this format was kept. The node is added as an object with the attributes: its unique numerical ID `node_id`, its position as x and y coordinates relative to

the canvas and an array of nodes adjacent to it on the graph `adjacent_nodes`. This was updated once my method for circle packing had been established to account for the circles of different radii encompassing terminal vertices and whether or not they had been appropriately scaled, with new attributes `vertex_radius` and a boolean `scaled`.

Moving

The state of a node moving relies on a check for the mouse being down and a node being in its vicinity. The node's position is updated to the mouse's as long as the mouse remains in the canvas.

Deletion

Node deletion was where I encountered the first indexing errors. To avoid these I added unique IDs to the node object's structure, and going forward ensured that all variable objects had an ID attribute.

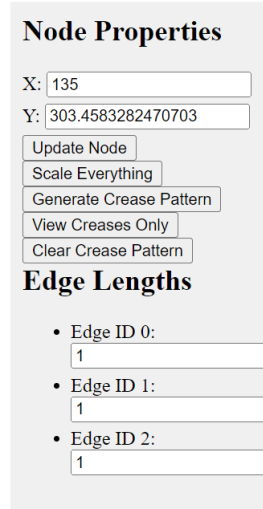
4.1.4 Connections

The variable name 'connection' is unfortunately vague, and doesn't explain what it represents well. There are several types of line in this crease pattern generator: the edges that define the planar graph, the active paths, the convex hull lines and the crease pattern lines. The variable `connection` only refers to the edges of the graph, and in retrospect should have been named 'edges'.

Connections are declared as objects and pushed to a `connections` array upon instancing, and are defined by their attributes: the ID of the connection, the two nodes it connects, and its length which can be changed by the user in the user menu.

4.1.5 User Menu

The user menu (see figure 1) allows the user to manually change the position of the nodes and the length of the connections, the latter of which is vital for changing the radii of a terminal node and therefore the circle packing formed. Later into the project, this menu was further updated to allow for generating a crease pattern, and changing the view to show just the crease pattern for printing purposes, or clearing the crease pattern, the idea inspired by TreeMaker's extensive menu options.



Node Properties

X:

Y:

Edge Lengths

- Edge ID 0:
- Edge ID 1:
- Edge ID 2:

Figure 1: User menu

4.2 Convex Hull

In a crease pattern, there exists a convex hull for the set of terminal vertices. The edge information for this hull is useful for the following reasons: the edges will always be major creases, and by comparing the path length of an edge to its actual length errors in the tree design can be easily found.

Towards the end of the project it became apparent that computing the hull was redundant for finding creases, as the lines created by polygons overlap with the hull. In the latter case, I unfortunately didn't consider error checking the tree design element to be within the scope of the project so it is not implemented. This functionality is kept, as it turned out to be a helpful visual aid, and in light of future additions.

4.2.1 Quickhull method

In evaluating which method to use for computing a convex hull, I referenced the Wikipedia article on convex hull algorithms [10], taking into account the time complexity and how much time I had to implement it. I used the Quickhull algorithm, which has a good expected time complexity of $O(n \log n)$ where n is the number of vertices, and followed the tutorial available on the website geeksforgeeks [11]. An overview of the Quickhull algorithm for a 2-D

plane is as follows:

1. Let the set of points in a 2-D space be S
2. Let P_{min} be a point in S with the minimum x
3. Let P_{max} be a point in S with the maximum x
4. Let the line L be defined by P_{min} and P_{max}
5. Let P_{new} be the point furthest from the line L
6. Form a triangle with the points P_{min} , P_{max} and P_{new}
7. Discard any points interior to the triangle from the set S
8. Recurse with new points (P_{min}, P_{new}) and (P_{new}, P_{max}) until the set S is empty

This leaves you with the set of lines that form a convex polygon around all the terminal vertices. The edges of the hull are represented with yellow lines, and are updated with every mouse movement event (see figure 2).

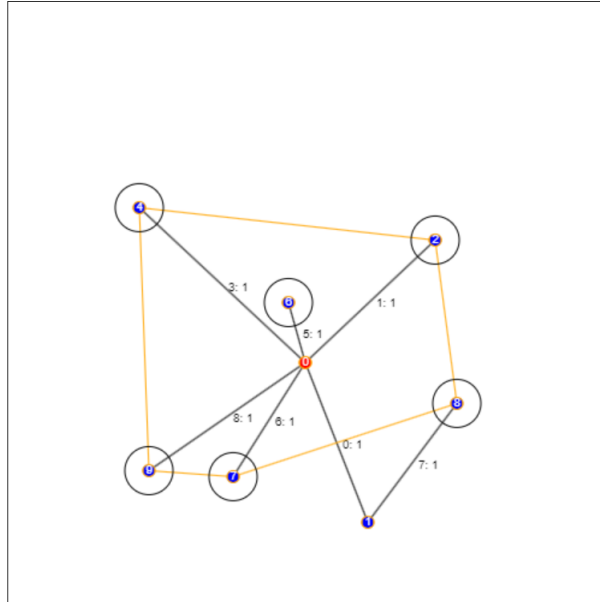


Figure 2: Convex hull

4.3 Circle Packing: Compacting circles

Whether or not a finite set of circles can be packed into a unit square is NP-hard, as Erik D. et al. proved in their paper Circle Packing for Origami Design Is Hard [12], and furthermore does not consider rivers in the packing. For these reasons, the method of packing assumes that the input set of circles can always be packed into a square, and there are no error checking capabilities.

For the method of packing, I considered a physics based approach at first, as the circles:

- Should not deviate too far from their relative positions to one another
- Do not necessarily need to be in the most efficient packing arrangement
- Should take into account rivers for the packing arrangement

My initial idea was to use a physics simulator such as `matter.js`, where each circle would be treated as a two-dimensional inelastic ball affected by gravity, dropped into a square-shaped "bucket" of pre-determined side length dependent on the total area of the circles. However, as the side length of the square cannot be accurately determined since the goal is not to obtain the most efficient packing, as well as it being an inflexible method for considering rivers, I opted for a different approach which did not require a physics simulator which involves compacting spheres in a shrinking enclosure. This method is - as far as I am aware - not described as a method of circle packing in any media.

The method of compacting circles is as follows:

- Each terminal node has a radius attribute equivalent to the path length of the edge connected to it.
- The positions of the terminal nodes relative to the canvas are transferred to a second canvas of the same size
- Each terminal node is represented as a circle, and its radius is its radius attribute multiplied by a scale factor of arbitrary size
- The rivers are also transferred to this canvas, as initially each river is defined by the start and end terminal nodes as well as the shortest path

length between the two; this data is converted appropriately so that they are represented by the circles instead, and their length scaled by the same factor as the terminal nodes

- A square of side length equivalent to the canvas size shrinks around the circles
- There are collision checks run with every decrement of the square's side length, in the following order:
 1. Checks are run to ensure that the distance between circles separated by a river are greater than or equal to the (scaled) length of the river
 2. Each pair of circles are checked for an overlap, and adjusted using the angle their centres are from one another and how much they overlap
 3. The circle centre positions are checked whether they lie within the square, and are adjusted appropriately
- These collision checks are run through a while loop, with two conditions to end: no overlaps are detected, or either 100 or 2000 repeats - dependant on whether there are rivers or not, respectively - of the loop have run, indicating that a suitable arrangement cannot be found and therefore the square size is at a minimum.
- The arrangement and sizes of the circles are then transferred back to the original canvas, with the square representing the size of the canvas

This solution is currently unoptimised, and has an estimated time complexity of n^2 for circle overlaps, and tends to an exponential for river checks. Furthermore, several issues arise in ensuring the square shrinks with the circles staying inside it. Increasing the upper limit for the while loop increases the chance that the circles stray outside of the square, while decreasing it increases the chance that the square will stop shrinking prematurely; the numbers 100 and 2000 were obtained through trial and error, and work well for most arrangements.

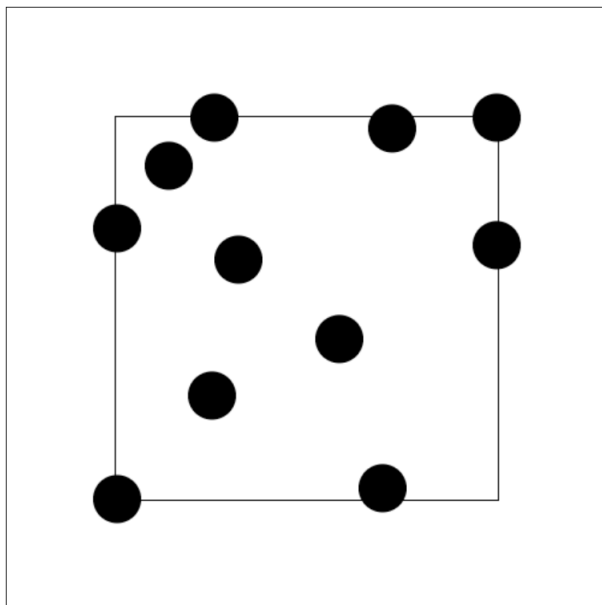


Figure 3: Compacting circles in a star graph (midway)

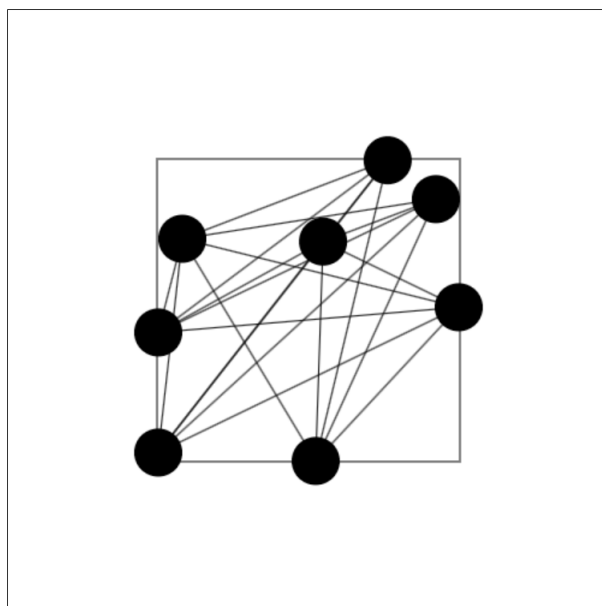


Figure 4: Compacting circles in a river graph (midway)

The lines in figure 4 represent the rivers between nodes. Introducing river checks increased the instances of circles escaping the square, so post-shrinkage another check is run to see how many circles are outside the square, and the square is increased in size by the average distance the circles are from the square's edges.

4.4 Finding major creases

4.4.1 Finding polygons

The edges of the polygons formed by the graph are creases. The method for finding the smallest polygons formed in the graph can be reworded as "finding all the chordless cycles in a planar graph", and a method rooted in graph theory is described in Marcel Wild's article on generating cycles, but I had trouble implementing what was described in Wild's paper as the graph formed by the circle packing was not necessarily connected [13].

As the graph was represented in a coordinate space, I took advantage of this instead. The method I used is repeated for every node pair:

1. A line L_1 is formed between the node pair (n_1, n_2)
2. For each of the nodes stemming off of n_2 , a line L_2 is formed between n_2 and the node stemming from it, n_3
3. The angle between n_2 and n_3 is compared against the minimum angle, which begins at 181°
4. Step 3 is repeated until all the nodes have been checked, and the line L_2 which formed the smallest angle becomes the next edge of the polygon
5. These steps are repeated, with n_1 and n_2 being replaced with the two previous nodes, until n_2 is the same as the original n_1 .

Similar to the circle packing method, this is also extremely unoptimised but does not cause any performance issues. Due to the redundant repetitions, the array of polygons is also run through a function to remove duplicates, and then triangles and quadrilaterals are identified. An example of this method working in theory is illustrated in figure 5.

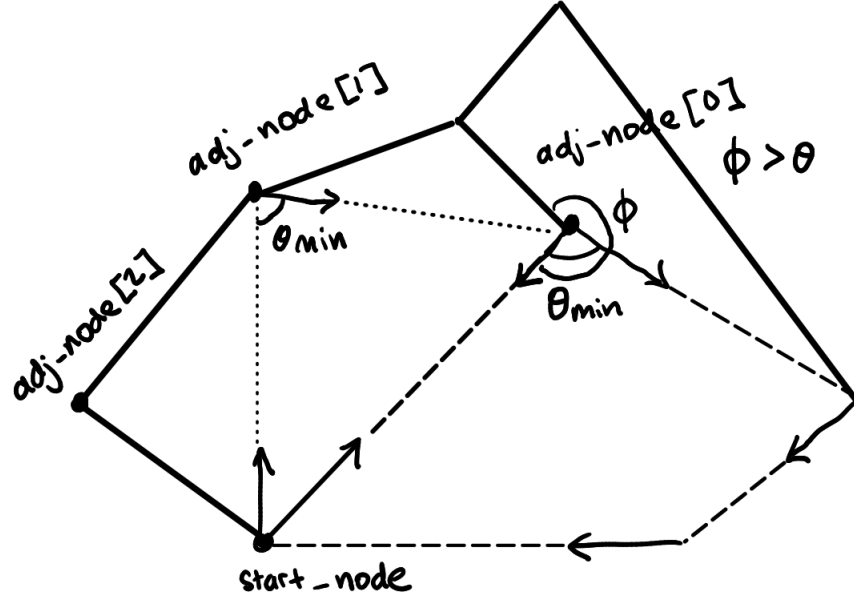


Figure 5: Example: Finding a polygon

I did not account for rivers when making this function, and as such, crease patterns that require rivers are not fully generated due to missing polygons.

4.4.2 Active Paths

The line formed between any two circles tangential to one another is an active path, and therefore a major crease [14]. Due to the inaccuracies of the circle packing algorithm, some circles either overlap slightly, or have gaps between them. The check for the distance between two circles relative to their combined radii accounts for this by increasing the range in which a path is accepted as active.

4.4.3 Other lines of interest

The lines formed by the convex hull are naturally also creases, and should be the same type of crease as well. The polygon finder overlaps with these however, so it is redundant.

4.5 Finding sub-bases

Finding the creases in the sub-bases proved the most difficult, and while the triangles were relatively easy to implement, I did not find a neat solution for the quadrilaterals, and instead resorted to using numerical analysis, resulting in too many inaccuracies that caused it to not work properly. For this reason, I have left out the quadrilateral sub-base functionality.

4.5.1 Triangles

For active triangles, the only creases are the lines passing through each vertex and the triangle's incentre, and the perpendiculars from the incentre to each side of the triangle. The incentre of a triangle can be calculated with the following equation, where the vertices of the triangles are denoted by position vectors A, B and C:

$$I = \frac{A |\overline{BC}| + B |\overline{CA}| + C |\overline{AB}|}{|\overline{BC}| + |\overline{CA}| + |\overline{AB}|}$$

For an inactive triangle, the perpendiculars are not dropped from the incentre. Instead, they meet the side of the triangle at the points where the circles intersect with the edge, which incidentally is the same for an active triangle, but for an active triangle this point coincides with the perpendicular dropped from the incentre. This means for each inactive edge there are two extra creases, as shown in figure 6.

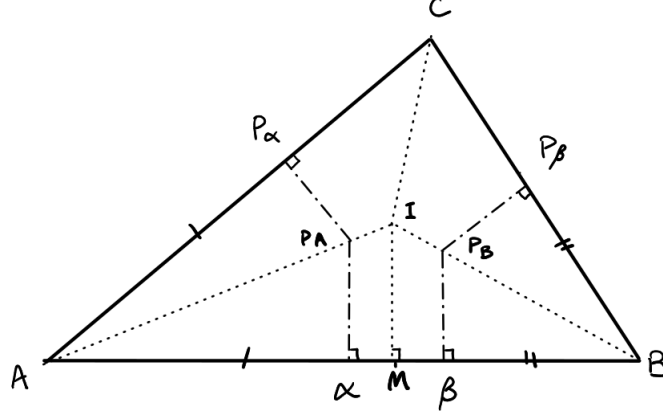


Figure 6: Inactive triangle creases

To reference figure 6, the following information is known:

- The points, represented as vectors from the origin: $\overline{OA}$, $\overline{OB}$, $\overline{OC}$.
- The radii of the circles with centres A, B and C, represented as magnitudes of vectors: $|\overline{A\alpha}|$ and $|\overline{B\beta}|$
- The point

We want to find the points P_A , P_B , P_α and P_β , as vectors from the origin. We can work out α , β and M, the midpoint between α and β :

$$\overline{O\alpha} = \overline{OA} + \frac{|\overline{A\alpha}|}{|\overline{BA}|} \overline{BA} \quad (1)$$

$$\overline{O\beta} = \overline{OB} + \frac{|\overline{B\beta}|}{|\overline{AB}|} \overline{AB} \quad (2)$$

$$\overline{OM} = \overline{O\alpha} + \frac{1}{2} \overline{\alpha\beta} \quad (3)$$

We can then substitute these, along with the vector equation for I, the in-centre, to find the relevant vectors:

$$\begin{aligned}
\overline{OP_A} &= \overline{OA} + \overline{AP_A} \\
&= \overline{OA} + \frac{|\overline{A\alpha}|}{|\overline{AM}|} \overline{AI} \\
\overline{OP_\alpha} &= \overline{OA} + \overline{AP_\alpha} \\
&= \overline{OA} + \frac{|\overline{A\alpha}|}{|\overline{AM}|} \overline{AC}
\end{aligned}$$

and similarly,

$$\begin{aligned}
\overline{OP_B} &= \overline{OB} + \frac{|\overline{B\beta}|}{|\overline{BM}|} \overline{BI} \\
\overline{OP_\beta} &= \overline{OB} + \frac{|\overline{B\beta}|}{|\overline{BM}|} \overline{BC}
\end{aligned}$$

These position vectors are then saved as relevant pairs and pushed in the *crease* array.

4.5.2 Quadrilaterals

There are three types of quadrilateral sub-bases: The Husimi molecule, the Maekawa molecule and the Lang's gusset quad molecule. They are not mutually exclusive however, as Lang's gusset quad is a generalised form of the quadrilateral sub-base. There are two equations which can be constructed to find the inset quadrilateral, triangle, line or point inside the molecule:

$$|AA_{AB}| + |A'C'| + |CC_{CB}| = |AC| \quad (4)$$

$$|AA_{AB}| + |A'C'| + |CC_{CB}| = |AC| \quad (5)$$

where A, B, C and D are vertices of the quadrilateral and A', B', C' and D' are vertices of the inset quadrilateral, as shown in figure 7, and the equations can be used to solve - in most cases - for the distance h.

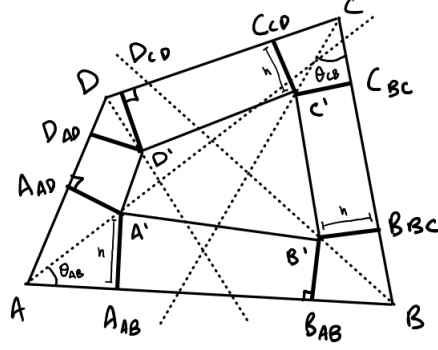


Figure 7: Lang's gusset quad molecule

While I did not obtain a useful equation, the final equations I ended up with are as follows:

$$\begin{aligned}\overline{AA_{AB}} &= \frac{h}{|\overline{AB}| \tan \theta_{AB}} \overline{AB} \\ \overline{CC_{CB}} &= \frac{h}{|\overline{CB}| \tan \theta_{CB}} \overline{CB} \\ \overline{A'C'} &= \overline{A'A} + \overline{AC} + \overline{CC'}\end{aligned}$$

Which can be expanded to

$$\begin{aligned}\overline{A'C'} &= \frac{h}{|\overline{AB}|} \left[(B_x - A_x) \cot \theta_{AB} - (B_y - A_y) \right] \\ &\quad + \begin{bmatrix} C_x - A_x \\ C_y - A_y \end{bmatrix} \\ &\quad + \frac{h}{|\overline{CB}|} \left[(B_x - C_x) \cot \theta_{CB} - (B_y - C_y) \right]\end{aligned}$$

The gusset quadrilateral is a special case of the universal molecule, an incredibly useful construction which I regrettably did not explore in this project.

4.6 Significant helper functions

4.6.1 Getting the shortest path

Retrieving the shortest path between two nodes is important for finding active paths and for finding rivers, and in general is a useful function. It uses a depth-first search, which is efficient for tree graphs.

```
function get_path(start_node, end_node) {
  const visited = new Set();
  const stack = [{ node: start_node, path: [] }];
  while (stack.length) {
    const { node, path } = stack.pop();
    if (node === end_node) {
      return path.concat(node);
    }
    if (!visited.has(node)) {
      visited.add(node);
      node.adjacent_nodes.forEach((adj_node) => {
        stack.push({ node: adj_node,
                      path: path.concat(node) });
      });
    }
  }
  return null;
}
```

The DFS algorithm begins at the start node, exploring along each branch until reaching a leaf node before backtracking. The visited nodes are tracked in a set variable `visited`, and an array `stack` is initialised, containing the start node and an empty path.

During each iteration of the while loop, the last stack item is removed and stored in `node` and `path`. Two checks are run: if the goal node has been found the path is returned; the second if decides where the algorithm checks next.

If the current node hasn't been visited before, `visited` is updated and its adjacent nodes along with the path, is added to the stack. If it has been visited before, then the algorithm backtracks, the last items of the stack

being popped until it reaches an unvisited node. Otherwise the loop ends and nothing is returned, indicating no paths between the two nodes, which in the case of this graph would be an error.

4.6.2 Removing duplicates in an array

This was incredibly useful throughout, and I made it following an issue that arose with finding polygons. Normal filtering methods did not work for an array of arrays, as arrays are compared by reference instead of content in JavaScript, e.g. `[1, 2, 3] === [1, 2, 3]` would return `false` in most cases.

```
function remove_duplicates_in_array(array) {
  const seen = new Set();
  const stringify_and_sort = (a) =>
    a.slice().sort().join(',');

  const unique_arrays = array.filter((a) => {
    const sorted_string = stringify_and_sort(a);
    if (seen.has(sorted_string)) {
      return false;
    } else {
      seen.add(sorted_string);
      return true;
    }
  });
  return unique_arrays;
}
```

This uses the very helpful Set data type, which stores unique objects and does not add an element to its list if it already exists in the set. The arrays are then converted to strings, as string comparisons are done by content in JavaScript, and traditional filtering by comparison is done, returning an array of unique arrays.

4.6.3 Linking two canvas elements

Running two scripts at once in JavaScript required asynchronous execution of the circle packing algorithm. A Promise object represents an operation that will definitely either succeed or fail in the future [15].

```
function loop() {  
  return new Promise((resolve, reject) => {  
    if (!is_square_minimum_size) {  
      update_canvas();  
      requestAnimationFrame(() => {  
        loop().then(resolve).catch(reject);  
      });  
    } else {  
      resolve({c: circles, s: square_side_length});  
    }  
  });  
}
```

In this method, a `Promise` is declared with the following operation. The `loop()` function is recursively called if the square has not shrunk to its minimum size, in the callback function passed to `requestAnimationFrame()` which is used to schedule a function to be called before the next visual update to prevent inconsistent frame rate. Otherwise, the `Promise` is immediately resolved.

The line `loop().then(resolve).catch(reject)` handles the resolution and rejection of the `Promise`, where `.then` specifies what happens upon resolution and `.catch` upon rejection. In this method rejection is not defined so nothing will happen, but for resolution the variables `circles` and `square_side_length` are returned to a function that converts then transfers them to the other canvas, therefore concluding the circle packing.

5 Final Results

5.1 Crease pattern generator: Walkthrough

Upon startup, the user is met with two empty canvases and the user menu.

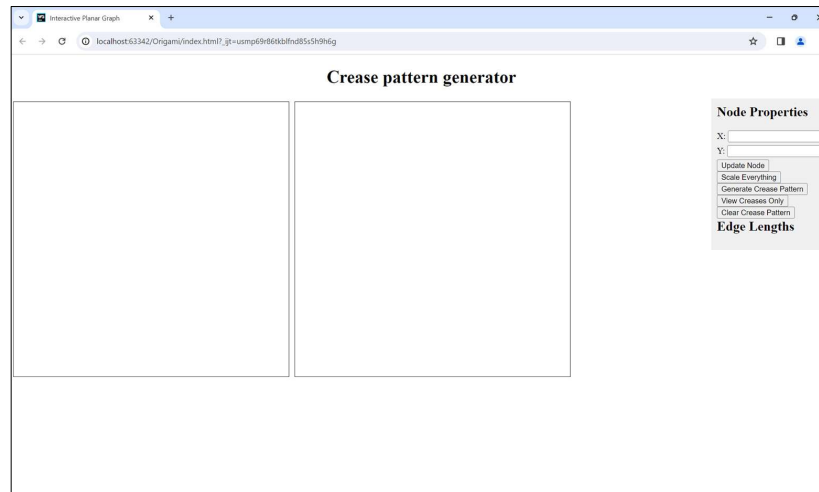


Figure 8: Startup screen

The user can then interact with the left canvas to input their tree design.

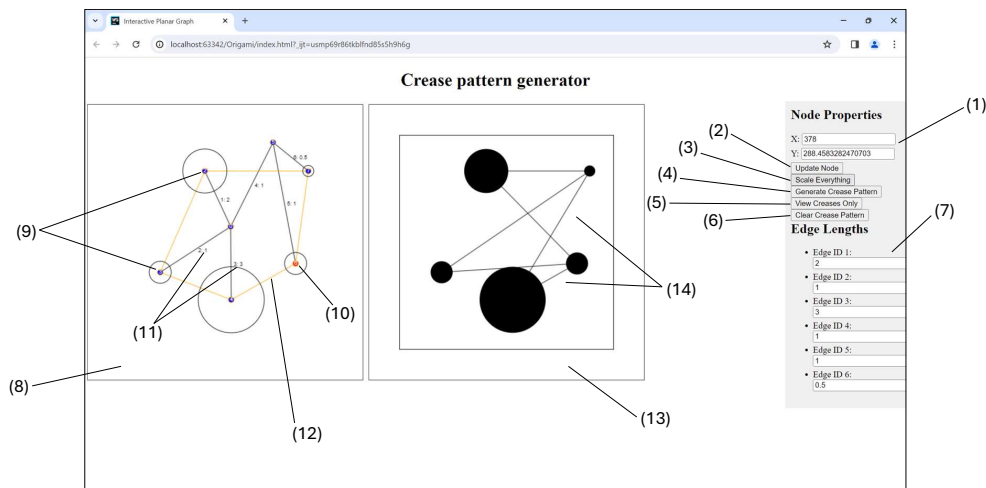


Figure 9: Tree input

In figure 9:

- (1) Input selected node's position
- (2) Updates the node's position after input
- (3) Commences circle packing
- (4) Generates a crease pattern based on the user input tree
- (5) Hides everything on the left canvas except for the crease lines
- (6) Clears the `creases` array and unhides other lines if hidden
- (7) Manually change edge length (and thus radii)
- (8) Left-click for adding/selecting nodes; Right-click for deleting nodes
- (9) Blue circles are unselected; node ID in white in the centre of each node
- (10) Red circles are currently selected
- (11) Edges format {edge ID : edge length}
- (12) Yellow convex hull lines
- (13) Circle packing canvas - cannot directly interact with
- (14) River lines

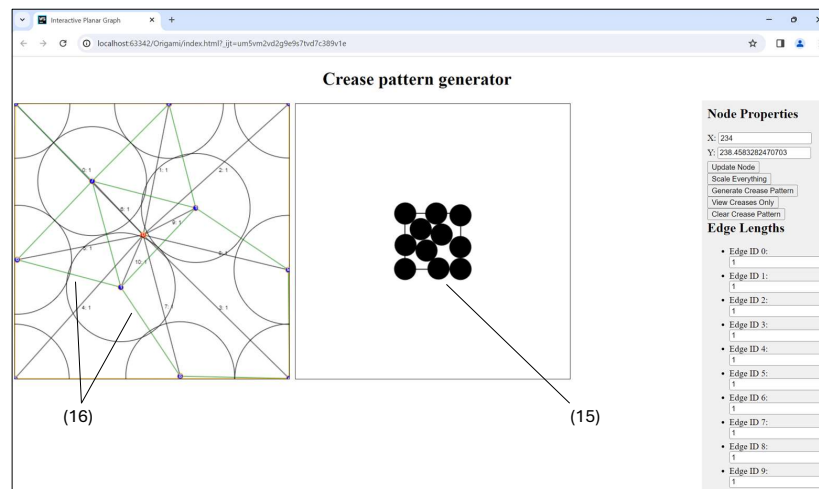


Figure 10: Scaling everything & Circle packing

In figure 10:

- (15) Circle packing arrangement complete
- (16) Green lines are active paths

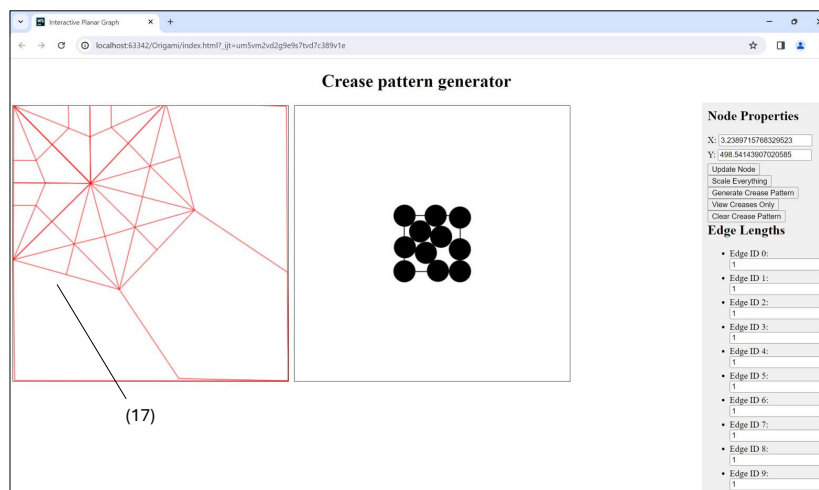


Figure 11: Crease pattern output

In figure 11:

- (17) Red lines are crease lines; there is no differentiation between mountain and valley folds

As previously mentioned, there are no methods for calculating the creases for a molecule greater than a triangle, which explains the large gaps in the crease pattern generated in figure 11.

5.2 Comparison with original goals

Throughout this project, I have become increasingly confident using JavaScript. For a couple of functions I used tutorials available online such as the quick-hull method, which helped me understand how recursion might work in JavaScript, and using DFS for finding paths in a tree graph. For the rest of the code, I used various sources ranging from videos to books to consulting my peers to aid the learning process.

In regards to the maths of origami, I still have a lot to learn. I understand the underlying theory behind the tree method and how circle packing aids it, but in terms of putting it into practice, I have only successfully implemented crease finding for a triangle molecule, and understanding how to find the creases for a universal molecule would be a useful next step for this project.

The main goal however - to create a web-based crease pattern generator - was successful. The program is interactive and while the options limited, can produce repeatably useful results given the right constraints. I have printed out a couple of crease patterns, one of which successfully folded into a bird base, and the other of which also folded into a usable base with 4 extrapolated creases. These are shown below:

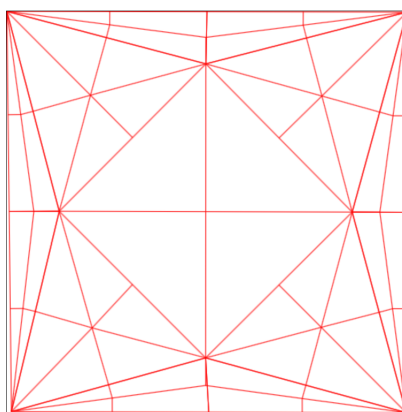


Figure 12: Crease pattern for five-node base

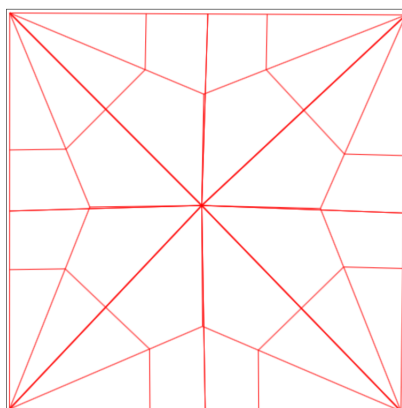
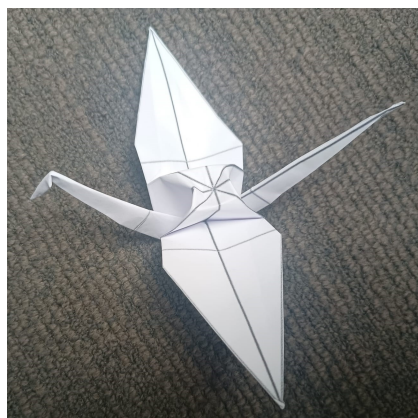
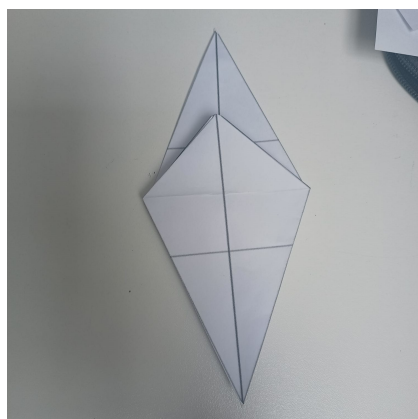
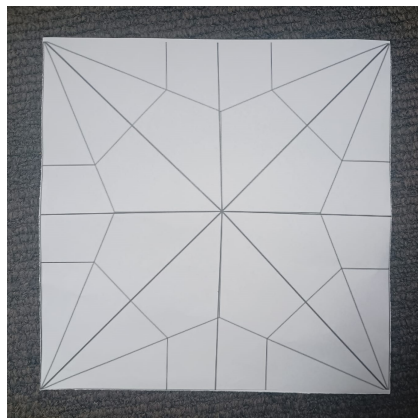
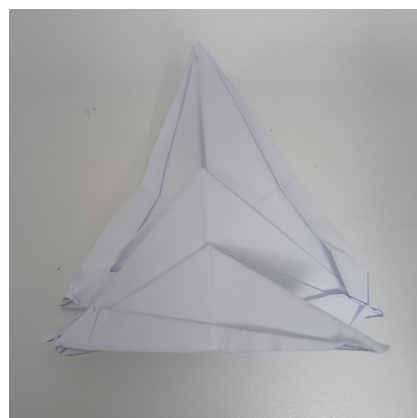
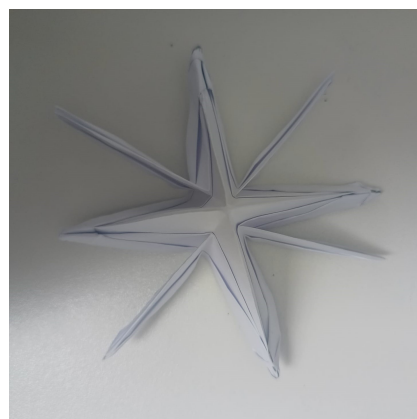
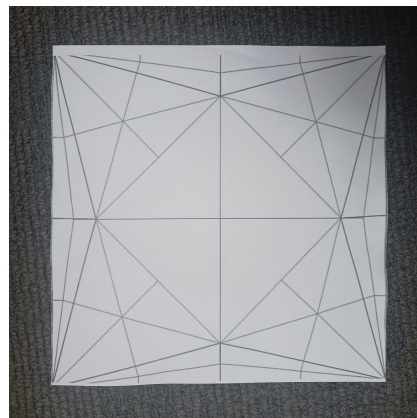


Figure 13: Crease pattern for six-node base

5-node crease



6-node crease



5.3 Comparison with TreeMaker

The program I created is in a lot of ways a very limited imitation of Robert Lang's TreeMaker, so I decided to best understand its flaws and where it can improve on, comparing it against certain aspects of this official crease pattern generator would be helpful.

The circle packing algorithm works very well for star graphs, almost always finding the most efficient packing while retaining their respective positions on the canvas, as seen in figure 14. When there are rivers involved however, the packing becomes more unstable, with less circles resulting in less accurate packings, as apparent in figure 15.

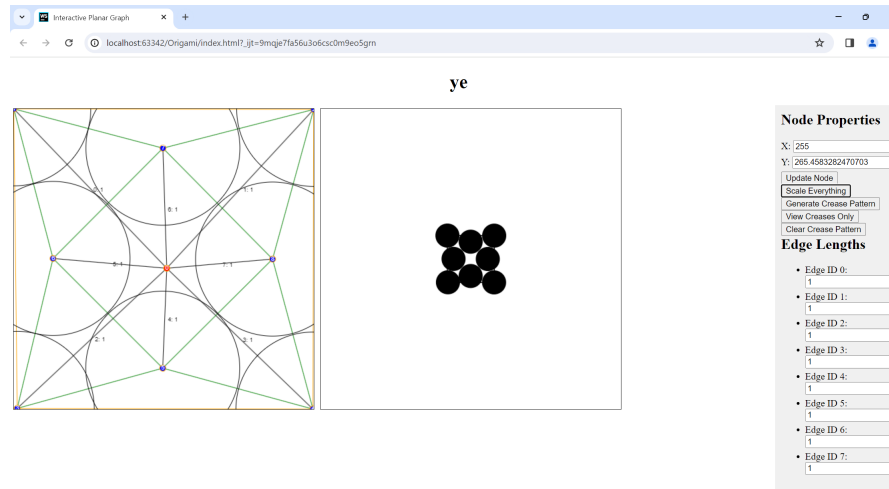


Figure 14: Accurate packing

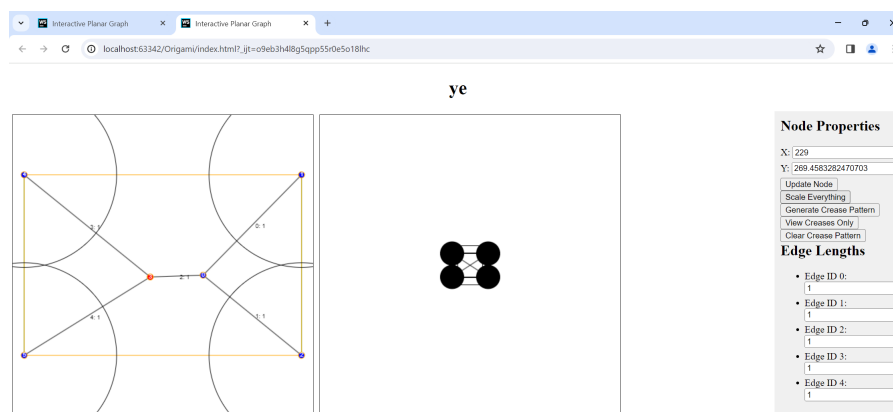


Figure 15: Inaccurate packing

Crease pattern generation works almost flawlessly for symmetrical crease patterns generated from a star graph as shown in figure 12 and figure 13. However, as the creases for a quadrilateral molecule isn't implemented, this greatly limits the crease patterns that can be generated. Furthermore as mentioned in the previous section, when rivers are involved not all polygons are found, again limiting the number of sub-bases and thus creases found. For example, in figure 16 the packing is repeatably accurate despite the rivers, however not all the polygons have been found.

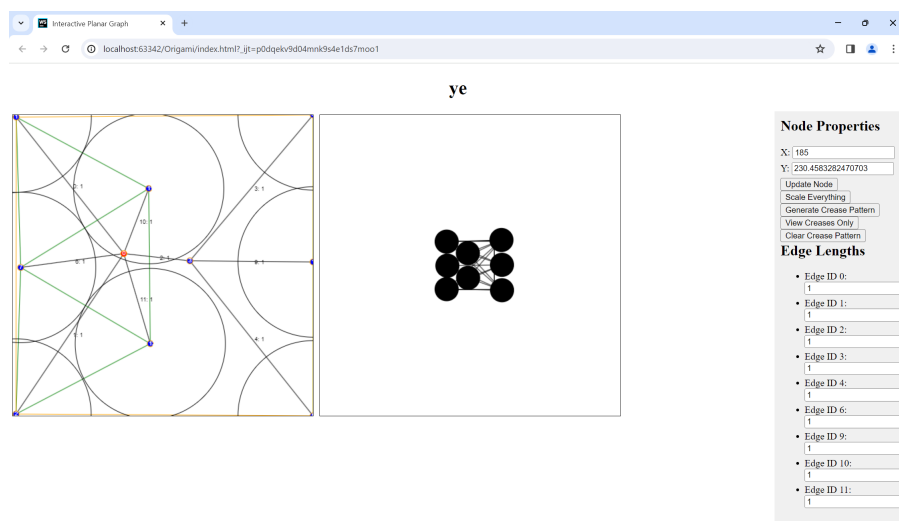


Figure 16: River packing lacking some polygons

Finally, there are many factors that I did not consider to be part of the scope of this project, such as error checking the initial tree that a user designs, finding the creases for a universal molecule - a major oversight - and extra attributes such as strain. I also did not implement any methods that would group the creases into valley folds and mountain folds, so the folding is left up to intuition. These are all reflected in the small number of options available in the menu, and the limited capabilities of the crease generator I created.

5.4 Final Tests

To test whether this program was useful to origami artists, I surveyed two of my peers in our school's origami club and three artists with less experience in origami, who used this program to generate crease patterns and create origami models. The results are as follows:

	Ease of Use	Use for Structured Designs	Use for Original Designs	Overall
Very unsatisfactory	0	1	0	1
Unsatisfactory	1	1	1	3
Neutral	0	1	1	2
Satisfactory	1	2	2	5
Very satisfactory	3	0	1	4

Table 2: Survey Results

In conclusion, the crease pattern generator was generally well received, and while its limitations proved unsurprisingly frustrating for creating "structured" designs that relied on bases similar to traditional ones, they did not hinder the creation of "original" designs, as accuracy was not the main focus, and the interactive element was appreciated on the whole.

A Definitions

To make this paper more concise, I've compiled together the definitions of key terminology, taken from various sources with some modified slightly to fit their purposes in this project [6]:

- Crease pattern: A set of straight lines on a 2D plane, which don't intersect; each line represents either a valley or a mountain fold; can also be thought of as a graph on a 2D planar surface
- Convex hull: A set of straight lines connecting the outermost points in a dataset, which form a convex polygon (all the angles in the polygon are less than 180 degrees)
- Planar graph: A connected graph without any crossing edges
- Flap: A part of a folded model that can freely rotate along one axis, with one end not connected to the rest of the model and the other connected to the model via a "hinge", in the same way that a door is to a wall
- River: A section of the folded model that connects two (or more) flaps, without being a flap itself, e.g. the body of an origami lizard, which connects 2 pairs of 3 flaps
- Active path: A connection between two nodes where actual distance between them on the plane is the same as the sum of the edge lengths of the shortest path on the tree
- Active polygon: A polygon on the tree where every edge is an active path
- Uniaxial base: An origami base that, when flat-folded, all of its flaps lie on a single axis with the hinge creases all perpendicular to that axis
- Internal nodes: nodes which are connected to 2 or more nodes
- Terminal nodes: nodes which are only connected to one other node a.k.a "leaf" nodes
- Base: The basic, foundational structure of an origami model, defined by the number, size and positions of its flaps
- Subbase/molecule: A base within a base, which when collapsed does

not affect the foldability of the main base

- Flat foldable: A 3D origami model that can be folded onto a 2D plane while retaining its structure, and back again.
- Stack: a data-structure that follows the last-in first-out principle (LIFO)

References

1. Shūsui S. Senbazuru Oriката. 1797
2. Rozenberg L. On the Evolution of the Notation System. 2019. Available from: <https://origamiusa.org/thefold/article/evolution-notation-system>
3. Lang R.J. TreeMaker. 2015. Available from: <https://langorigami.com/article/treemaker/>
4. AbrashiOrigami. Polygon Packing Method Introductory Course. 2021. Available from: https://www.youtube.com/playlist?list=PL2imMO_S0rwuGUbXdGjjasoZ4Un_pP4c
5. Wikipedia. Yoshizawa–Randlett system. 2011-present. Available from: <https://langorigami.com/article/treemaker/>
6. Martin A. Origami Mathematics. 2019. Available from: <https://core.ac.uk/download/pdf/212801758.pdf>
7. Lang R.J. Origami Design Secrets: Mathematical Methods for an Ancient Art, Second Edition. 2003
8. Lang R.J. TreeMaker. Version 5.0.1. 2016 Apr 13. Available from: <https://github.com/bugfolder/TreeMaker>
9. Kraft R. Rabbit Ear. Version 0.9.32. 2022. Available from: <https://rabbitear.org/>
10. Wikipedia. Convex hull algorithms. 2007-present. Available from: https://en.wikipedia.org/wiki/Convex_hull_algorithms
11. Yagni A. Quickhull Algorithm for Convex Hull. 2024. Available from: <https://www.geeksforgeeks.org/quickhull-algorithm-convex-hull/>
12. Demaine ED, Fekete SP, and Lang R.J. Circle Packing for Origami Design is Hard. 2010. Available from: https://erikdemaine.org/papers/DiskPacking_Origami5/paper.pdf
13. Wild M. Generating all cycles, chordless cycles, and Hamiltonian cycles with the principle of exclusion by Marcel Wild. 2008. Available from: <https://www.sciencedirect.com/science/article/pii/S1570866707000020>
14. Lang R.J. The Tree Method of Origami Design. Available from: https://langorigami.com/wp-content/uploads/2015/09/TreeMkr40jp_s.pdf

15. MDNDevelopers. Introducing asynchronous JavaScript. 2024. Available from: <https://developer.mozilla.org/en-US/docs/Learn/JavaScript/Asynchronous/Introducing>